

LAPORAN MACHINE LEARNING
Analisis dan Implementasi Algoritma Supervised Learning



Disusun Oleh:

Nama : Ali Akbar Bermano
NPM : G1A024041
Kelas : A

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Machine Learning merupakan cabang kecerdasan buatan yang memungkinkan komputer belajar dari data tanpa harus diprogram secara eksplisit. Bab kedua dari buku "Introduction to Machine Learning with Python" karya Andreas C. Müller dan Sarah Guido membahas berbagai algoritma supervised learning yang menjadi fondasi penting dalam praktik machine learning.

Supervised learning adalah jenis pembelajaran mesin di mana model dilatih menggunakan data berlabel untuk memprediksi output pada data baru. Bab ini memperkenalkan berbagai algoritma klasifikasi dan regresi, mulai dari yang sederhana seperti k-Nearest Neighbors (k-NN) hingga yang kompleks seperti Neural Networks (MLP), serta teknik-teknik seperti Decision Trees, Random Forest, Gradient Boosting, Support Vector Machines, dan model linear.

Pemahaman mendalam tentang setiap algoritma sangat penting karena masing-masing memiliki kekuatan dan keterbatasan yang berbeda. Faktor-faktor seperti jumlah data, jumlah fitur, jenis masalah (klasifikasi atau regresi), dan kebutuhan interpretabilitas model mempengaruhi pilihan algoritma yang tepat.

Dalam project ini, dilakukan implementasi dan analisis berbagai algoritma supervised learning menggunakan Python dan pustaka scikit-learn. Berbagai dataset sintetis (forge, wave, two_moons, make_blobs) dan dataset nyata (Breast Cancer Wisconsin, Boston Housing) digunakan untuk mengevaluasi performa masing-masing algoritma. Setiap algoritma dievaluasi pada kemampuan generalisasi, risiko overfitting/underfitting, serta pengaruh hyperparameter terhadap performa model.

1.2 Tujuan

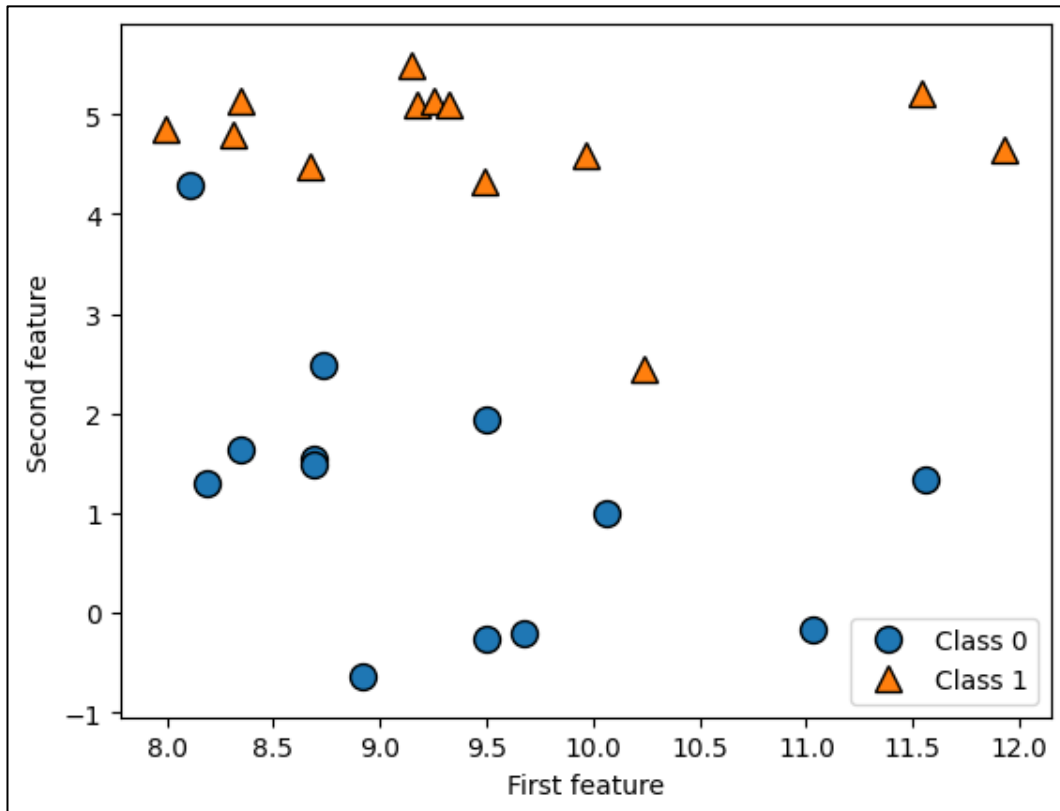
1. Memahami cara kerja dan implementasi berbagai algoritma supervised learning menggunakan Python.
2. Menganalisis performa model k-NN, Linear Models, Naive Bayes, Decision Trees, Random Forest, Gradient Boosting, SVM, dan Neural Networks pada berbagai dataset.
3. Memahami pengaruh hyperparameter terhadap kompleksitas model dan risiko overfitting/underfitting.
4. Memahami kapan dan mengapa penskalaan fitur diperlukan untuk algoritma tertentu.
5. Menginterpretasikan output prediksi model termasuk decision function dan probabilitas prediksi.
6. Mengembangkan kemampuan praktis dalam implementasi machine learning menggunakan scikit-learn.

BAB II

METODOLOGI DAN OUTPUT EKSPERIMEN

2.1. Persiapan dan Dataset

Sebelum memulai eksperimen, pustaka yang diperlukan diinstall dan diimport terlebih dahulu. Pustaka utama yang digunakan meliputi numpy, pandas, matplotlib, dan mglearn (pustaka pendukung buku). Konfigurasi awal meliputi pengaturan parameter visualisasi dan presisi angka.



Dataset forge adalah dataset sintetis untuk klasifikasi biner yang terdiri dari 26 sampel dengan 2 fitur. Scatter plot menunjukkan distribusi dua kelas yang akan dipisahkan oleh berbagai algoritma klasifikasi. Dataset ini sering digunakan karena ukurannya kecil sehingga mudah divisualisasikan.

Dataset wave adalah dataset sintetis untuk regresi yang terdiri dari 40 sampel dengan satu fitur kontinu. Target nilainya bersifat kontinu sehingga cocok digunakan untuk mengevaluasi algoritma regresi. Sumbu horizontal menunjukkan nilai fitur sedangkan sumbu vertikal menunjukkan nilai target.

2.2. Dataset Nyata: Breast Cancer dan Boston Housing

Dataset Breast Cancer Wisconsin adalah dataset klasifikasi biner nyata yang terdiri dari 569 sampel dengan 30 fitur. Setiap sampel mewakili pengukuran sel tumor, dan targetnya adalah label "malignant" (212 sampel) atau "benign" (357 sampel). Dataset ini sangat umum digunakan dalam benchmark algoritma klasifikasi.

Dimensi
X.shape: (26, 2)

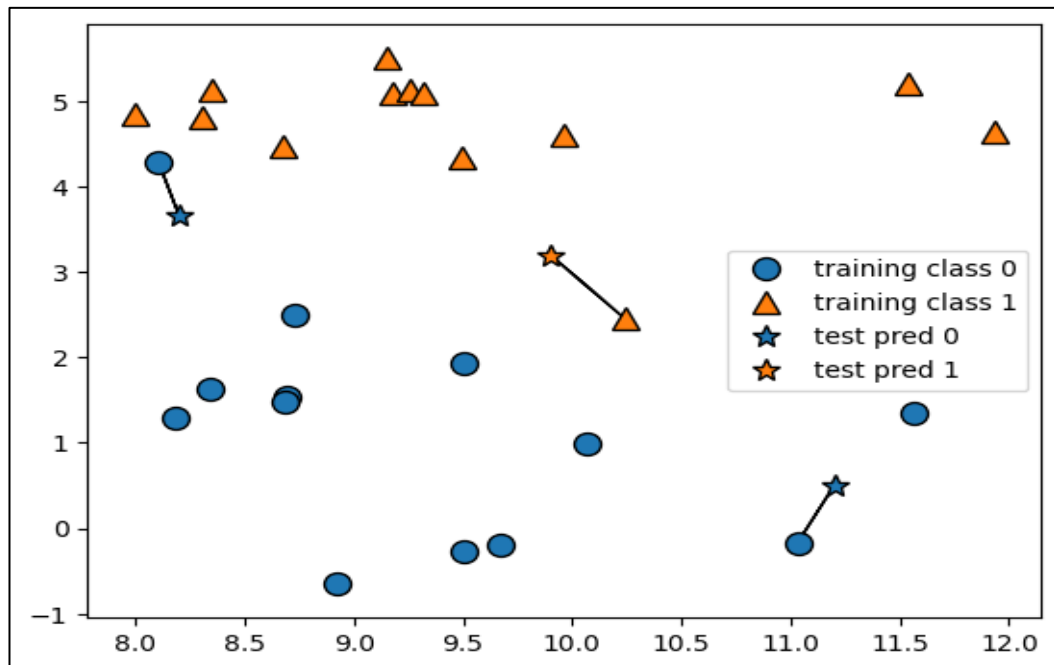
data:

Dataset Boston Housing (dimuat melalui `mglearn` karena sudah dihapus dari `scikit-learn` ≥ 1.2 karena alasan etis) terdiri dari 506 sampel dengan 13 fitur yang mendeskripsikan karakteristik kawasan perumahan. Target variabelnya adalah harga median rumah dalam ribuan dolar. Dataset extended Boston yang dihasilkan melalui feature engineering memiliki 104 fitur (hasil perkalian antar pasangan fitur asli).

```
Boston
cancer.keys():
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
Housing:
```

2.3. Algoritma k-Nearest Neighbors (k-NN)

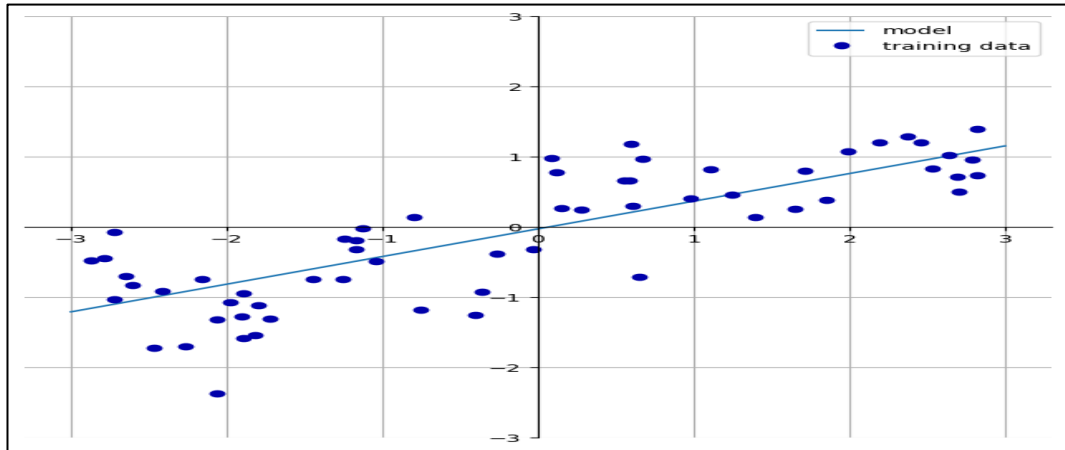
k-Nearest Neighbors (k-NN) adalah algoritma supervised learning yang memprediksi label kelas berdasarkan mayoritas label dari k tetangga terdekat pada ruang fitur. Algoritma ini tidak membentuk model secara eksplisit (lazy learning) – ia menyimpan seluruh data latih dan melakukan perhitungan saat prediksi.



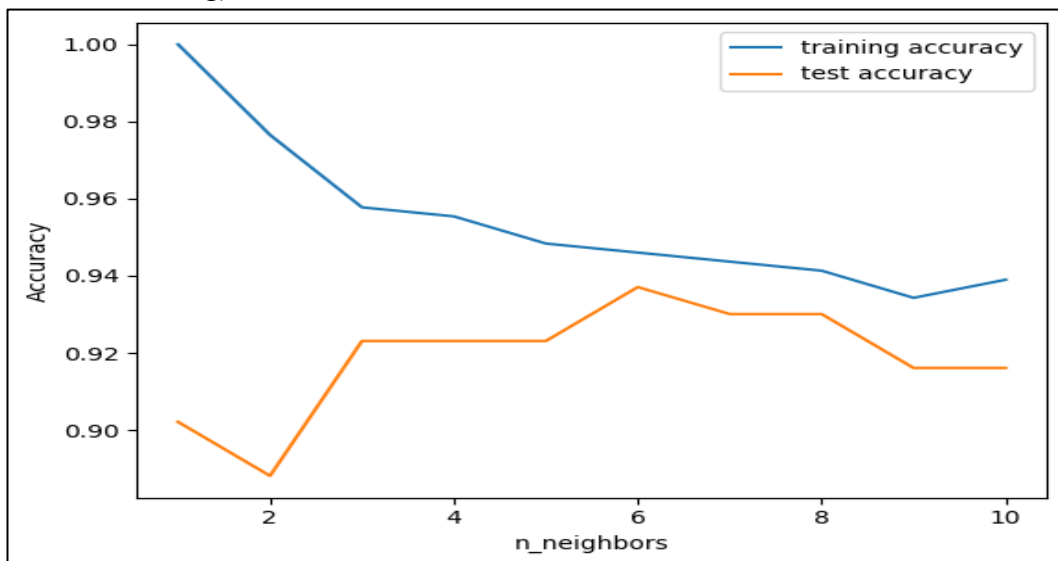
Gambar di atas menunjukkan hasil prediksi dari model *K-Nearest Neighbors* pada data uji (*test data*). Titik lingkaran biru merepresentasikan kelas pelatihan 0, sedangkan titik segitiga oranye merepresentasikan kelas pelatihan 1. Titik bintang (bintang biru untuk prediksi kelas 0 dan bintang oranye untuk prediksi kelas 1) menunjukkan data uji beserta garis penghubung yang mengarah ke tetangga terdekatnya dalam ruang fitur. Visualisasi ini memperlihatkan bagaimana model menentukan kelas suatu data uji berdasarkan kedekatannya dengan titik-titik data pelatihan di sekitarnya. Untuk kasus regresi, k-NN memprediksi nilai target sebagai rata-rata nilai target dari k tetangga terdekat. Dengan $k=1$, prediksi berfluktuasi tajam mengikuti setiap titik data (overfitting). Dengan $k=3$, prediksi lebih halus namun tetap mengikuti pola umum data.

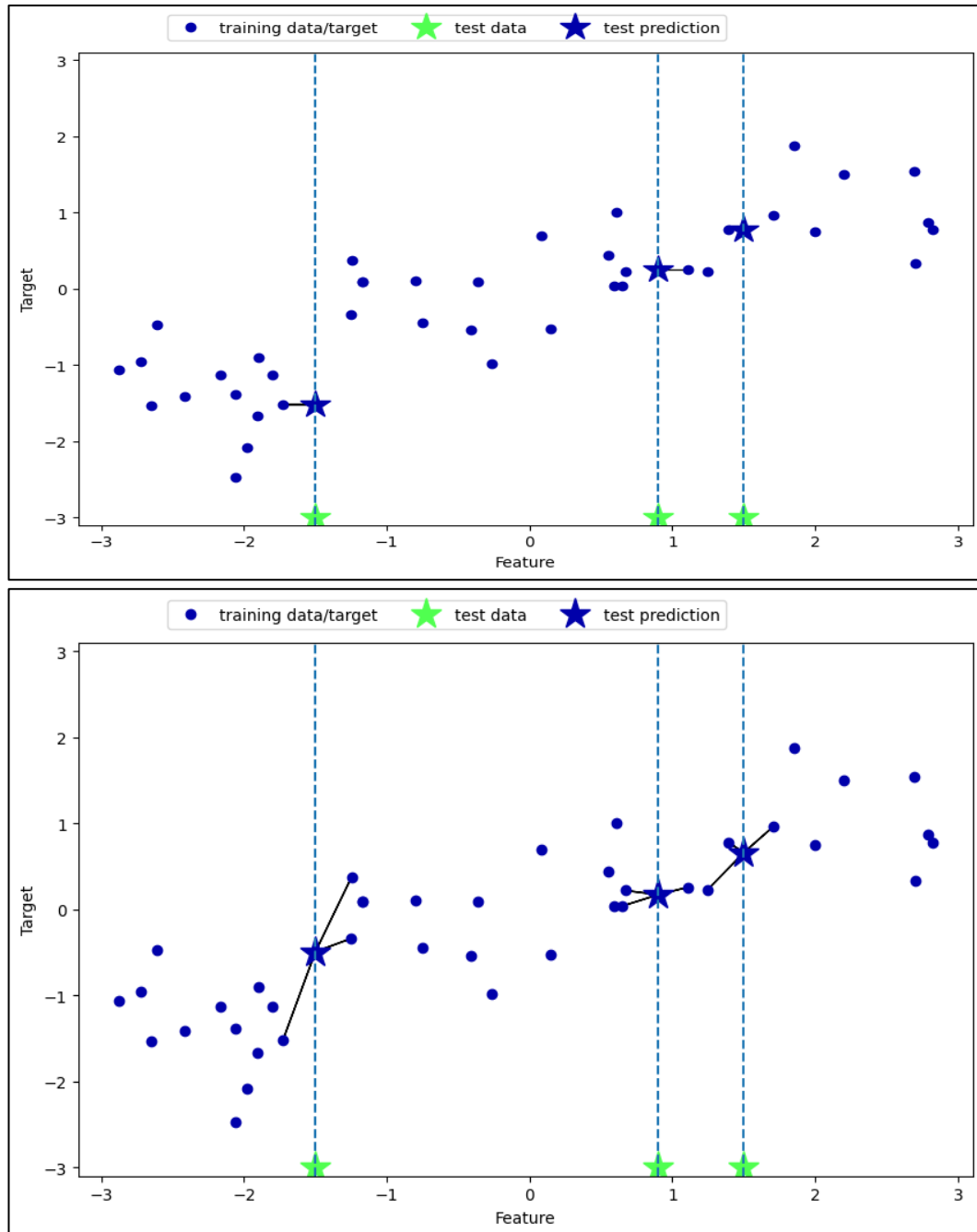
2.4. Model Linear (Linear Models)

Model linear membuat prediksi menggunakan kombinasi linear dari fitur-fitur input. Untuk klasifikasi, model linear mencari garis (atau *hyperplane* dalam dimensi tinggi) yang memisahkan kedua kelas. Untuk regresi, model mencocokkan sebuah garis lurus (atau *hyperplane*) pada data.



Ridge Regression menambahkan regularisasi L2 pada Linear Regression untuk mencegah overfitting. Parameter alpha mengontrol kekuatan regularisasi: alpha besar menghasilkan koefisien mendekati nol (model sederhana/underfitting), alpha kecil memberikan lebih banyak kebebasan pada koefisien (model kompleks, risiko overfitting).

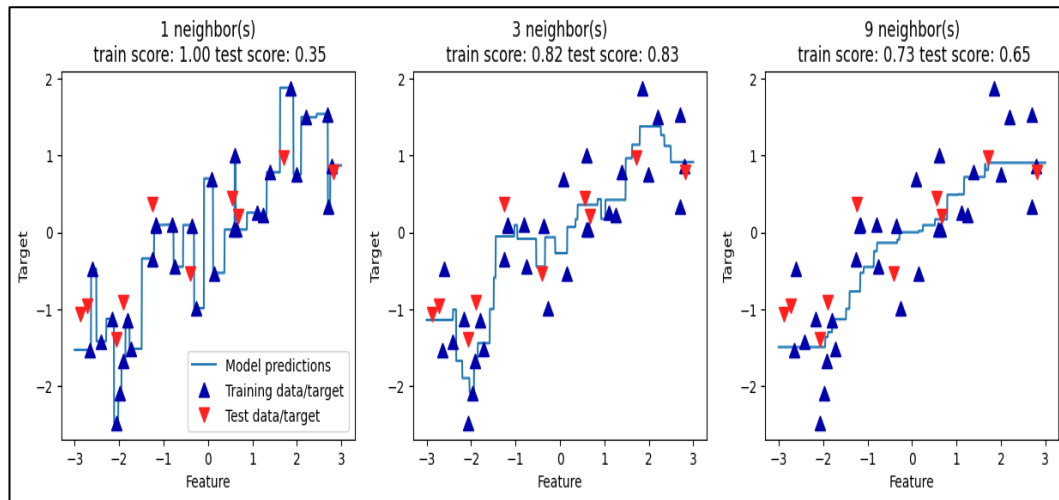




Gambar di atas mengilustrasikan mekanisme kerja algoritma K-Neighbors Regressor (*K-Nearest Neighbors* untuk masalah regresi) dalam memprediksi nilai target kontinu berdasarkan data uji (*test data*). Titik-titik lingkaran biru tua merepresentasikan data pelatihan (*training data*), bintang hijau di sumbu X merepresentasikan lokasi data uji, dan bintang biru tua merepresentasikan nilai prediksi yang dihasilkan oleh model (*test prediction*).

Pada grafik bagian atas ($k = 1$), model menentukan nilai prediksi hanya berdasarkan satu tetangga terdekat. Setiap titik data uji (bintang hijau) dihubungkan dengan satu garis lurus ke titik data pelatihan terdekatnya, dan nilai target dari titik terdekat tersebut langsung dijadikan nilai prediksinya.

Pada grafik bagian bawah ($k = 3$), model menggunakan tiga tetangga terdekat untuk melakukan prediksi. Setiap data uji dihubungkan dengan tiga garis ke tiga titik data pelatihan di sekitarnya, kemudian nilai prediksi diperoleh dari rata-rata (*average*) nilai target dari ketiga tetangga terdekat tersebut. Penggunaan nilai k yang lebih besar ini membantu menghaluskan hasil prediksi dan mengurangi sensitivitas model terhadap *noise* pada data latih.



2.5. Naive Bayes Classifier

Naive Bayes adalah keluarga model klasifikasi yang sangat cepat karena hanya mempelajari statistik setiap fitur per kelas secara terpisah. Pendekatan ini disebut "naive" karena mengasumsikan independensi antar fitur, yang sering kali tidak terpenuhi di dunia nyata. BernoulliNB cocok untuk fitur biner, GaussianNB untuk fitur kontinu, dan MultinomialNB untuk data dengan fitur non-negatif seperti frekuensi kata dalam text classification.

Dari output di atas, BernoulliNB menghitung berapa kali setiap fitur bernilai 1 pada setiap kelas. Informasi ini digunakan untuk mengestimasi probabilitas kondisional $P(\text{fitur} \mid \text{kelas})$ yang kemudian digabungkan menggunakan teorema Bayes untuk menghasilkan prediksi kelas.

2.6. Decision Trees (Pohon Keputusan)

Decision Tree membentuk model prediksi berupa struktur pohon dengan menentukan aturan "jika-maka" berdasarkan nilai fitur. Setiap node internal pohon berisi satu pertanyaan tentang fitur tertentu, dan setiap cabang merepresentasikan jawaban dari pertanyaan tersebut. Daun pohon berisi prediksi kelas atau nilai target.

Feature importance menunjukkan seberapa penting setiap fitur dalam membuat prediksi pada pohon keputusan. Fitur dengan nilai importance tinggi digunakan lebih sering dan lebih dekat ke akar pohon. Pada dataset Cancer, beberapa fitur seperti "worst radius" dan "worst concave points" memiliki importance yang sangat tinggi, sedangkan banyak fitur lainnya hampir tidak digunakan.

Salah satu keterbatasan Decision Tree untuk regresi adalah ketidakmampuannya berekstrapolasi ke luar rentang nilai data latih. Model hanya

dapat memprediksi nilai yang sudah ada pada data latih (berbentuk tangga horizontal). Sebaliknya, Linear Regression dapat berekstrapolasi karena menggunakan persamaan linear yang terus berlanjut.

2.7. Random Forest

Random Forest adalah metode ensemble yang menggabungkan banyak Decision Tree yang dibuat secara independen dengan cara bootstrap aggregation (bagging). Setiap pohon dilatih pada subset acak dari data latih dan hanya mempertimbangkan subset acak fitur pada setiap split. Prediksi akhir adalah mayoritas suara (klasifikasi) atau rata-rata (regresi) dari semua pohon.

Gambar menunjukkan batas keputusan dari 5 pohon individual (acak) dan hasil gabungannya. Setiap pohon memiliki batas keputusan yang berbeda-beda dan terlihat "kasar", tetapi hasil gabungan dari semua pohon menghasilkan batas keputusan yang jauh lebih halus dan mulus. Inilah keunggulan ensemble method dibandingkan satu model tunggal.

Feature importance dari Random Forest lebih menyebar dibandingkan satu pohon tunggal. Dengan 100 pohon, setiap fitur mendapat kesempatan untuk berkontribusi pada setidaknya beberapa pohon, menghasilkan estimasi importance yang lebih stabil dan reliabel. Fitur "worst radius" dan "worst concave points" tetap menjadi fitur terpenting.

2.8. Gradient Boosting

Gradient Boosting adalah metode ensemble yang membangun pohon secara berurutan, di mana setiap pohon baru berusaha memperbaiki kesalahan dari pohon sebelumnya. Berbeda dengan Random Forest yang membangun pohon secara paralel dan independen, Gradient Boosting membangun pohon secara iteratif dengan mengurangi residual error secara bertahap.

Feature importance dari Gradient Boosting menunjukkan distribusi yang lebih terpusat dibandingkan Random Forest – beberapa fitur diabaikan sepenuhnya (nilai 0). Ini terjadi karena Gradient Boosting lebih selektif dalam memilih fitur yang paling informatif untuk memperbaiki error residual. Parameter penting Gradient Boosting meliputi `n_estimators` (jumlah pohon), `learning_rate` (seberapa besar kontribusi setiap pohon), dan `max_depth` (kedalaman maksimal pohon).

2.9. Support Vector Machine (SVM)

Support Vector Machine (SVM) adalah algoritma klasifikasi yang berusaha menemukan hyperplane dengan margin maksimum yang memisahkan dua kelas. Titik-titik data yang paling dekat dengan hyperplane disebut support vectors. SVM dapat bekerja pada ruang dimensi tinggi dan menggunakan kernel trick untuk memisahkan data yang tidak linearly separable.

Grid 3x3 menunjukkan pengaruh parameter C (kekuatan regularisasi) dan gamma (lebar kernel RBF) terhadap batas keputusan SVM. Gamma kecil menghasilkan batas keputusan yang mulus (mempertimbangkan banyak titik), gamma besar menghasilkan batas yang sangat tidak teratur. C kecil memberikan lebih banyak toleransi terhadap misklasifikasi, C besar berusaha

mengklasifikasikan semua titik dengan benar meski menghasilkan batas yang lebih kompleks.

SVM sangat sensitif terhadap skala fitur. Boxplot menunjukkan bahwa fitur-fitur pada dataset Cancer memiliki rentang nilai yang sangat berbeda-beda (dalam skala logaritmik). Tanpa penskalaan, SVM mengalami performa yang buruk (akurasi test hanya 0.63). Setelah penskalaan min-max, akurasi melonjak drastis karena semua fitur berada dalam rentang $[0, 1]$.

2.10. Neural Networks (MLPClassifier)

Neural Network atau Multilayer Perceptron (MLP) adalah algoritma yang terinspirasi dari cara kerja neuron di otak manusia. MLP terdiri dari beberapa lapisan: input layer, satu atau lebih hidden layers, dan output layer. Setiap lapisan terhubung penuh ke lapisan berikutnya, dan setiap koneksi memiliki bobot yang dipelajari selama training.

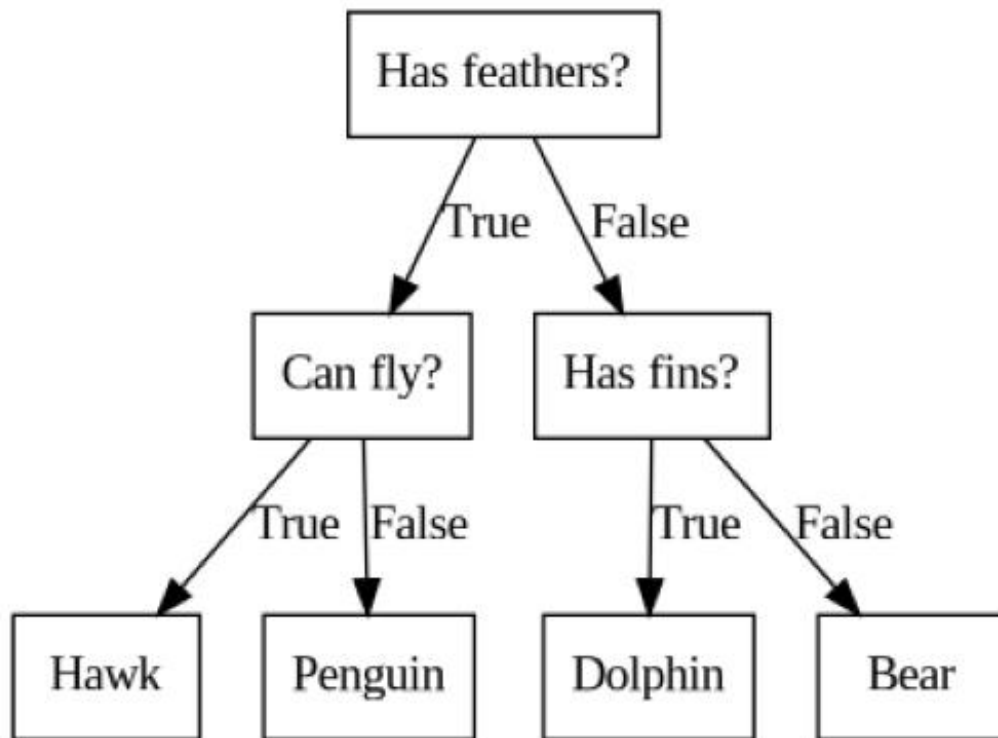
Fungsi aktivasi membuat neural network mampu mempelajari pola non-linear. ReLU (Rectified Linear Unit) adalah fungsi $\max(0, x)$ yang sangat populer karena komputasinya sederhana dan efektif. Tanh (hyperbolic tangent) menghasilkan output antara -1 dan 1, lebih smooth dibandingkan ReLU.

Perbandingan berbagai konfigurasi MLP menunjukkan bahwa jumlah unit hidden layer, jumlah layer, fungsi aktivasi, dan parameter regularisasi alpha semuanya mempengaruhi kompleksitas batas keputusan. MLP dengan aktivasi tanh dan dua hidden layer menghasilkan batas yang lebih mulus dibandingkan ReLU. Alpha yang lebih besar memberikan regularisasi lebih kuat dan batas yang lebih sederhana.

Heatmap bobot antara layer input dan hidden layer pertama memberikan gambaran tentang bagaimana neural network "melihat" data. Warna merah menunjukkan bobot positif (fitur meningkatkan aktivasi neuron), warna biru menunjukkan bobot negatif. Pola pada heatmap menunjukkan bahwa setiap neuron tersembunyi berfokus pada kombinasi fitur yang berbeda.

2.11. Ketidakpastian dalam Prediksi

Bab ini juga membahas cara menginterpretasikan kepercayaan diri (confidence) model dalam prediksinya. Scikit-learn menyediakan dua cara utama: `decision_function` (mengembalikan skor raw per kelas) dan `predict_proba` (mengembalikan probabilitas yang telah dinormalisasi antara 0 dan 1 untuk setiap kelas).



Decision function menghasilkan nilai kontinu yang sulit diinterpretasikan secara langsung karena skalanya bervariasi antar model dan dataset. Predict_proba lebih mudah dipahami karena nilainya selalu antara 0 dan 1 dan jumlahnya selalu 1 untuk setiap sampel. Gradasi warna pada predict_proba lebih jelas menunjukkan area ketidakpastian model (warna lebih muda = probabilitas sekitar 0.5, model tidak yakin).

BAB III

KESIMPULAN

3.1. kesimpulan

Berdasarkan eksperimen dan analisis yang dilakukan terhadap berbagai algoritma *supervised learning*, dapat disimpulkan bahwa tidak ada satu algoritma tunggal yang terbaik untuk semua jenis permasalahan. Algoritma sederhana seperti *k-Nearest Neighbors* (k-NN) dan pohon keputusan (*Decision Trees*) sangat baik untuk pemahaman awal dan mudah diinterpretasikan, tetapi cenderung mudah mengalami *overfitting* atau melambat pada dataset berdimensi tinggi. Penggunaan metode *ensemble* seperti *Random Forest* dan *Gradient Boosting* terbukti secara signifikan meningkatkan kinerja prediksi serta kemampuan generalisasi model dengan menggabungkan beberapa pohon keputusan.

Selain itu, eksperimen menunjukkan pentingnya tahapan prapemrosesan data; algoritma seperti *Support Vector Machines* (SVM) dan *Neural Networks* (MLP) sangat sensitif terhadap rentang nilai data, di mana penerapan penskalaan fitur terbukti mampu meningkatkan akurasi prediksi secara drastis. Pemilihan dan pengaturan *hyperparameter* (seperti nilai k pada k-NN, parameter regularisasi α atau C , serta kedalaman pohon) memegang peranan kunci dalam menjaga keseimbangan antara risiko *underfitting* dan *overfitting*. Terakhir, evaluasi ketidakpastian prediksi melalui fungsi probabilitas memberikan tingkat transparansi yang lebih tinggi dalam menilai keandalan keputusan yang dibuat oleh model.